

AI Prompts, Instructions, And Workflows

Purpose

This document records the project-specific AI prompts, engineering instructions, and workflows used to plan, implement, debug, document, and publish KOHLER AI Bathroom Designer. It does not reproduce private platform or system instructions. It describes the actionable project context and prompt patterns that shaped the repository.

1. Product Brief Prompt

```text

Build a browser-based AI bathroom designer. Accept room length, room width, budget, design language, and layout style. Select a feasible fixture collection from the catalog, enforce budget and room-space limits, place fixtures without overlap, and render the result in an interactive 3D bathroom scene.

```

Expected behavior

- Return a feasible collection rather than a marketing mockup.
- Use the catalog as the source of truth.
- Keep decisions explainable through cost, space, water, and fixture data.
- Make the first screen the working planning tool.

2. Backend Architecture Prompt

```text

Implement a FastAPI service with a clear API boundary. Load the bathroom catalog from CSV, score products by aesthetic and eco criteria, optimize selection with PuLP when available, fall back deterministically when the solver is unavailable, and return JSON-ready room and fixture placement data.

```

Workflow

1. Define the Pydantic request model.
2. Load and normalize catalog fields.
3. Score products for the selected design language.
4. Apply budget, category, space, and layout constraints.
5. Solve with PuLP/CBC.
6. Fall back to greedy selection on solver failure.
7. Convert selected records into placement payloads.
8. Return a stable API response.

3. Optimization Prompt

```text

Select at most one fixture per category. Keep total price within the requested budget and required clearance within 70% of the room area. Prefer products that match the requested theme and have strong aesthetic and eco scores. Use the same footprint measurement rule for selection constraints and physical placement.

```

Rules encoded by the workflow

- Room dimensions are bounded from 4 to 40 feet.
- Budget is zero or greater.
- Maximum allocated area is ``length_ft * width_ft * 0.70``.
- Categories are limited to one selected product each.
- Toilet centerline clearance is included during overlap checks.
- Linear Single-Wall uses the along-wall dimension in its feasibility constraint.
- Unsupported layout values use the linear fallback.

4. Frontend Experience Prompt

```text

Create the usable bathroom planning experience as the first screen. Use a compact project brief sidebar, a resolved-layout workspace, an interactive Three.js viewport, inspection HUD, preliminary specifications, metrics, and a bill of materials. Keep the visual language premium, architectural, and precise.

```

Interaction workflow

1. User enters dimensions and budget.
2. User selects one of five design languages.
3. User selects one of three layout styles.
4. User submits the brief.
5. The browser posts the request to ``/api/optimize``.
6. The response updates metrics and the BOM.
7. The scene is rebuilt from the returned room and fixture data.
8. The user can orbit, zoom, select a fixture, change finishes, and inspect water behavior.

5. Design Language Prompt

```text

Provide five coherent bathroom design languages with distinct material, lighting, wall, floor, and grout treatments. Keep them appropriate for a premium bathroom planner and make each choice visible in the 3D room.

```

Current choices

- Minimalist Modern
- Japanese Zen
- Classic Luxury
- Mediterranean Coastal
- Biophilic Sanctuary

6. Layout Prompt

```text

Provide three layout strategies that make physical sense for bathroom planning: Zoned Wet/Dry, Linear Single-Wall, and Parallel Galley. Keep placement wall-aware, non-overlapping, and consistent with the optimizer's footprint calculations.

```

Removed choices

Island Focal Point and Symmetry Axis were removed from the product because they did not belong in the final supported layout set. Documentation and UI must not reintroduce them without a product

decision.

7. Water Simulation Prompt

```text

Add an interactive water simulation for compatible fixtures. Confine water to sink and bathtub boundaries. Enable streams, ripples, caustics, steam, vortex motion, and splash droplets, but do not render bubbles or foam. Every optional effect must be null-safe when disabled and stopping the simulation must reset its state.

```

Debugging workflow

1. Trace the water toggle from the button or fixture HUD.
2. Inspect every optional effect for a null reference.
3. Ensure disabled effects are guarded in both `toggle()` and `update()`.
4. Check pool geometry against the fixture boundary.
5. Check particle positions against the configured basin radius.
6. Reset the pool level when simulation stops.
7. Confirm camera-distance auto-shutoff remains safe.

8. Visual Debugging Prompt

```text

When the screenshot shows translucent geometry obscuring the room, identify the actual mesh constructor rather than only reducing opacity. Remove or shorten oversized enclosure geometry while preserving the functional fixture and water hardware.

```

Applied resolution

The shower builder originally added full-height glass planes that dominated the camera view. The final workflow removes those oversized planes instead of relying only on transparency.

9. Error-Diagnosis Prompt

```text

Trace the reported failure to the nearest code path that directly controls it. State one falsifiable hypothesis, identify the cheapest validation check, make the smallest repair, and rerun focused diagnostics before expanding scope.

```

Examples

- Water crash: disabled foam and splash systems were still called without guards.
- Scene obstruction: full-height shower glass geometry was the controlling mesh.
- Empty product menu: the feature was removed from the current frontend scope.
- Git push rejection: the remote had an existing commit and required rebase.

10. Documentation Workflow Prompt

```text

Synchronize README, PRD, architecture, rules, design, and task documentation with the actual current code. Remove stale features and document setup through

```
both local Python and Docker workflows.
'''
```

### ### Documentation outputs

- `README.md`: setup, API, features, tests, and deployment.
- `prd.md`: product requirements and acceptance criteria.
- `Architecture.md`: modules, runtime flow, deployment, and data flow.
- `rules.md`: domain, placement, water, API, and deployment rules.
- `design.md`: visual, interaction, responsive, and accessibility rules.
- `task.md`: completed work, validation, maintenance, and release tasks.
- `PROMPTS.md`: this project-specific AI prompt and workflow record.

## ## 11. Validation Workflow

```
```text
```

1. Read the local implementation surface.
2. Form one local hypothesis about the requested behavior.
3. Make the smallest focused edit.
4. Run the cheapest relevant executable check.
5. Repair the same slice if the check exposes a local defect.
6. Run post-edit diagnostics.
7. Review documentation and repository state.

```
'''
```

Commands

```
```bash
```

```
python -m pytest -q
python smoke_test.py
python -m py_compile backend/main.py backend/optimizer.py
'''
```

Browser-level validation should additionally check:

- Layout generation.
- Theme changes.
- Layout changes.
- Fixture selection.
- Finish selection.
- Water start and stop.
- No bubble or foam rendering.
- No water overflow.
- No scene obstruction.
- Mobile viewport behavior.

## ## 12. Publishing Workflow

```
```text
```

1. Add a publish-safe .gitignore.
2. Inspect for secrets and generated environments.
3. Initialize or inspect Git history.
4. Add source, assets, tests, and documentation.
5. Add the MIT license when requested.
6. Commit with a descriptive message.
7. Add the GitHub remote.

8. Rebase if the remote contains an initial commit.
9. Push the main branch.
10. Verify repository visibility and tracked file contents.
- ...

13. Scope Boundary

The following are intentionally not documented as private system instructions:

- Hidden platform prompts.
- Provider-specific internal policies.
- Private chain-of-thought or hidden reasoning.
- Credentials, access tokens, or authentication data.

This file documents only the project-level prompts, requirements, workflows, decisions, and validation methods that are appropriate for repository publication.